

```
1 C:\Users\24006528\PyCharmMiscProject\.venv\Scripts\
  python.exe C:\Users\24006528\PyCharmMiscProject\
  script.py
2 QUESTÃO 1 - Divisão Simples Treino/Teste
3
4 matriz de confusão:
5 [[ 57   7]
6  [   7 100]]
7
8 acuracia : 0.9181
9 precisão : 0.9346
10 recall   : 0.9346
11 F1Score  : 0.9346
12 QUESTÃO 2
13
14 tamanhos dos conjuntos:
15 treino: 278 amostras (48.9%)
16 validação: 120 amostras (21.1%)
17 teste    : 171 amostras (30.1%)
18
19 desempenho no conjunto de validacao
20 acuracia : 0.9167
21 precisão : 0.9221
22 recall   : 0.9467
23 F1Score  : 0.9342
24
25 desempenho no conjunto de teste
26 matriz de confusão:
27 [[58   6]
28  [   9 98]]
29
30 acurácia : 0.9123
31 precisão : 0.9423
32 recall   : 0.9159
33 F1Score  : 0.9289
34 QUESTÃO 3
35
36 fold 1:
37 matriz de confusão:
38 [[39   4]
39  [   4 67]]
```

```
40 acurácia : 0.9298 / precisão : 0.9437 | Recall : 0.
    9437 / F1Score : 0.9437
41
42 fold 2:
43 matriz de confusão:
44 [[32 11]
45  [ 4 67]]
46 acurácia : 0.8684 / precisão : 0.8590 | Recall : 0.
    9437 / F1Score : 0.8993
47
48 fold 3:
49 matriz de confusão:
50 [[34  8]
51  [ 5 67]]
52 acurácia : 0.8860 / precisão : 0.8933 | Recall : 0.
    9306 / F1Score : 0.9116
53
54 fold 4:
55 matriz de confusão:
56 [[40  2]
57  [ 5 67]]
58 acurácia : 0.9386 / precisão : 0.9710 | Recall : 0.
    9306 / F1Score : 0.9504
59
60 fold 5:
61 matriz de confusão:
62 [[37  5]
63  [ 3 68]]
64 acurácia : 0.9292 / precisão : 0.9315 | Recall : 0.
    9577 / F1Score : 0.9444
65
66 tabela de resultados por fold ---
67      Acurácia  Precisão    Recall  F1-Score
68 Fold
69 1      0.929825  0.943662  0.943662  0.943662
70 2      0.868421  0.858974  0.943662  0.899329
71 3      0.885965  0.893333  0.930556  0.911565
72 4      0.938596  0.971014  0.930556  0.950355
73 5      0.929204  0.931507  0.957746  0.944444
74
75 média das métricas
```

```

76 acurácia : 0.9104
77 precisão : 0.9197
78 recall   : 0.9412
79 F1Score  : 0.9299
80 QUESTÃO 4
81
82 --- Tabela Comparativa ---
83           Acurácia  Precisão  Recall  F1-
      Score
84 Estratégia

85 Divisão Simples    0.9181    0.9346  0.9346    0.
    9346
86 Holdout            0.9123    0.9423  0.9159    0.
    9289
87 K-Fold (média)     0.9104    0.9197  0.9412    0.
    9299
88
89 --- Análise das Matrizes de Confusão ---
90
91 Q1 (Divisão Simples):
92 [[ 57   7]
93  [   7 100]]
94
95 Q2 (Holdout - conjunto de teste):
96 [[58   6]
97  [   9 98]]
98
99 Q3 (K-Fold - soma das matrizes de cada fold):
100 [[182  30]
101  [   21 336]]
102
103 --- Tabela Comparativa ---
104           Acurácia  Precisão  Recall  F1-
      Score
105 Estratégia

106 Divisão Simples    0.9181    0.9346  0.9346    0.
    9346
107 Holdout            0.9123    0.9423  0.9159    0.
    9289

```

File - script

```
108 K-Fold (média)      0.9104    0.9197  0.9412    0.
    9299
109
110 --- Análise das Matrizes de Confusão ---
111
112 Q1 (Divisão Simples):
113 [[ 57   7]
114  [  7 100]]
115
116 Q2 (Holdout - conjunto de teste):
117 [[58   6]
118  [  9 98]]
119
120 Q3 (K-Fold - soma das matrizes de cada fold):
121 [[182  30]
122  [ 21 336]]
123
124 Process finished with exit code 0
125
```

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, KFold,
StratifiedKFold
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score,
    precision_score, recall_score, f1_score
)

print("QUESTÃO 1 - Divisão Simples Treino/Teste")

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.3, random_state=42, stratify=y)

clf1 = DecisionTreeClassifier(random_state=42)
clf1.fit(X_train, y_train)

y_pred1 = clf1.predict(X_test)

cm1 = confusion_matrix(y_test, y_pred1)
acc1 = accuracy_score(y_test, y_pred1)
prec1 = precision_score(y_test, y_pred1)
```

```

rec1    = recall_score(y_test, y_pred1)
f1_1    = f1_score(y_test, y_pred1)

print(f"\nmatriz de confusão:\n{cm1}")
print(f"\naccuracia : {acc1:.4f}")
print(f"precisão : {pre1:.4f}")
print(f"recall : {rec1:.4f}")
print(f"F1Score : {f1_1:.4f}")

print("QUESTÃO 2")

X_trainval, X_test2, y_trainval, y_test2 = train_test_split(X, y,
test_size=0.30, random_state=42, stratify=y)

X_train2, X_val2, y_train2, y_val2 = train_test_split(
    X_trainval, y_trainval,
    test_size=0.30,
    random_state=42,
    stratify=y_trainval
)

print(f"\ntamanhos dos conjuntos:")
print(f"treino: {len(X_train2)} amostras
({len(X_train2)/len(X)*100:.1f}%)")
print(f"validação: {len(X_val2)} amostras
({len(X_val2)/len(X)*100:.1f}%)")
print(f"teste : {len(X_test2)} amostras
({len(X_test2)/len(X)*100:.1f}%)")

clf2 = DecisionTreeClassifier(random_state=42)
clf2.fit(X_train2, y_train2)

y_pred_val2 = clf2.predict(X_val2)
print(f"\ndesempenho no conjunto de validacao")
print(f"acuracia : {accuracy_score(y_val2, y_pred_val2):.4f}")
print(f"precisão : {precision_score(y_val2, y_pred_val2):.4f}")
print(f"recall : {recall_score(y_val2, y_pred_val2):.4f}")
print(f"F1Score : {f1_score(y_val2, y_pred_val2):.4f}")

y_pred2 = clf2.predict(X_test2)

cm2    = confusion_matrix(y_test2, y_pred2)
acc2   = accuracy_score(y_test2, y_pred2)
prec2  = precision_score(y_test2, y_pred2)
rec2   = recall_score(y_test2, y_pred2)
f1_2   = f1_score(y_test2, y_pred2)

print(f"\ndesempenho no conjunto de teste")
print(f"matriz de confusão:\n{cm2}")
print(f"\naccurácia : {acc2:.4f}")
print(f"precisão : {prec2:.4f}")
print(f"recall : {rec2:.4f}")
print(f"F1Score : {f1_2:.4f}")

print("QUESTÃO 3")

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

fold_results = []
cms_kfold = []

```

```

for fold, (train_idx, test_idx) in enumerate(skf.split(X, y),
start=1):
    X_tr, X_ts = X[train_idx], X[test_idx]
    y_tr, y_ts = y[train_idx], y[test_idx]

    clf_k = DecisionTreeClassifier(random_state=42)
    clf_k.fit(X_tr, y_tr)
    y_pred_k = clf_k.predict(X_ts)

    cm_k = confusion_matrix(y_ts, y_pred_k)
    acc_k = accuracy_score(y_ts, y_pred_k)
    prec_k = precision_score(y_ts, y_pred_k)
    rec_k = recall_score(y_ts, y_pred_k)
    fl_k = f1_score(y_ts, y_pred_k)

    fold_results.append({
        "Fold": fold,
        "Acurácia": acc_k,
        "Precisão": prec_k,
        "Recall": rec_k,
        "F1-Score": fl_k
    })
    cms_kfold.append(cm_k)

    print(f"\nfold {fold}:")
    print(f"matriz de confusão:\n{cm_k}")
    print(f"acurácia : {acc_k:.4f} / precisão : {prec_k:.4f} | "
          f"Recall : {rec_k:.4f} / F1Score : {fl_k:.4f}")

df_folds = pd.DataFrame(fold_results).set_index("Fold")
means = df_folds.mean()

print("\ntabela de resultados por fold ---")
print(df_folds.to_string())
print("\nmédia das métricas")
print(f"acurácia : {means['Acurácia']:.4f}")
print(f"precisão : {means['Precisão']:.4f}")
print(f"recall : {means['Recall']:.4f}")
print(f"F1Score : {means['F1-Score']:.4f}")

acc3 = means['Acurácia']
prec3 = means['Precisão']
rec3 = means['Recall']
fl_3 = means['F1-Score']

print("QUESTÃO 4")

comparativo = pd.DataFrame({
    "Estratégia": ["Divisão Simples", "Holdout", "K-Fold (média)"],
    "Acurácia": [acc1, acc2, acc3],
    "Precisão": [prec1, prec2, prec3],
    "Recall": [rec1, rec2, rec3],
    "F1-Score": [fl_1, fl_2, fl_3]
}).set_index("Estratégia")
print("\n--- Tabela Comparativa ---")
print(comparativo.to_string(float_format="{:.4f}".format))

print("\n--- Análise das Matrizes de Confusão ---")
print(f"\nQ1 (Divisão Simples):\n{cm1}")
print(f"\nQ2 (Holdout - conjunto de teste):\n{cm2}")
print(f"\nQ3 (K-Fold - soma das matrizes de cada

```

```
fold):\n{sum(cms_kfold)})")

print("\n--- Tabela Comparativa ---")
print(comparativo.to_string(float_format="{:.4f}".format))

print("\n--- Análise das Matrizes de Confusão ---")
print(f"\nQ1 (Divisão Simples):\n{cm1}")
print(f"\nQ2 (Holdout - conjunto de teste):\n{cm2}")
print(f"\nQ3 (K-Fold - soma das matrizes de cada fold):\n{sum(cms_kfold)})")

"""
a) As três abordagens produziram métricas similares (~91-94%), mas a divisão simples pode ser otimista por depender de uma única partição aleatória.

b) A matriz do K-Fold (soma dos folds) cobre todos os dados e é mais representativa; as de Q1 e Q2 refletem apenas uma divisão específica.

c) A validação evita que o teste vazze informações para o ajuste do modelo, tornando a avaliação final mais honesta e reduzindo o risco de overfitting.

d) Para este dataset (569 amostras), o StratifiedKFold (K=5) é a estratégia mais adequada: usa todos os dados, mantém a proporção das classes e fornece estimativas mais estáveis do desempenho real do modelo.
"""
```